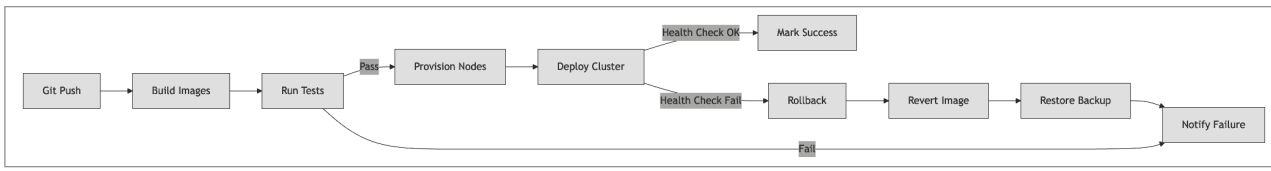

Figure 2: Fault-tolerant CI/CD state machine with rollback mechanisms

Edge orchestration

Traditional Kubernetes is heavyweight for edge use. Lightweight Kubernetes variants like KubeEdge and K3s have been proposed for resource-limited environments. KubeEdge extends K8s for IoT by offloading data handling to edge nodes. However, it still relies on etcd, adding overhead on small devices. In contrast, K3s is explicitly designed for the edge: it removes etcd in favor of SQLite, reducing footprint on ARM boards.

Edge provisioning

Automation tools like Ansible have been applied to edge clusters. Ansible has been used to bootstrap Raspberry Pi nodes for IoT, showing improved consistency. Integration of CI/CD into edge deployments has also been explored; Jenkins has been combined with Docker Swarm for distributed apps. Our work extends these ideas by combining Jenkins pipelines with Ansible playbooks for multi-node provisioning and orchestration, tailored to camera-equipped Pi 4 devices.

Monitoring and logging

Observability on edge devices remains challenging. Prometheus is a *de facto* standard for metrics collection. For log aggregation, Grafana Loki is a modern solution that works alongside Prometheus. Container-level tools like cAdvisor (Google) and Node Exporter (Prometheus project) are widely used for resource metrics collection. We leverage this stack in a unified manner: each Pi runs Node Exporter, cAdvisor, and Promtail to feed the central Prometheus/Loki servers, consistent with best practices for edge observability.

Edge vision and machine learning

MobileNetV2 is a lightweight convolutional neural network well-suited for mobile/embedded vision tasks, and bidirectional LSTMs can capture temporal patterns in video. Our violence detection model combines these components (fine-tuned on domain-specific data) to enable efficient inference on the Pi 4 CPU. Prior works on violence detection often use similar architectures, but our contribution is deploying such a model in a near-real-time edge pipeline with comprehensive orchestration and monitoring.

System design

Our framework integrates three core layers (Figure 1) that coordinate to deliver low-latency edge inference.

Edge layer: Comprises Raspberry Pi 4 nodes (4GB RAM) with cameras, running:

- *Dual-container pods:*
  - Detector container: Executes MobileNetV2+BiLSTM model
  - Watchdog container: Implements alerting pipeline
- *Observability agents:* Node Exporter (9100), cAdvisor (10250), Promtail (3100)

Control plane: Hosted on AWS EC2 t2.micro:

- K3s server with SQLite backend
- Camera-aware scheduling (camera=true node selector)
- Secure credential injection via Kubernetes Secrets

Monitoring and CI/CD layer: On-premises server hosting:

- *Observability stack:* Prometheus, Loki, Grafana (3000)
- *Automation:* Jenkins pipeline with Ansible integration

Operational workflow

The system executes through six coordinated phases:

- ***CI/CD pipeline:*** Automated image builds and credential injection
- ***Edge provisioning:*** Device optimisation and cluster joining
- ***Resource-aware scheduling:*** Camera-enabled node selection
- ***Inference and alerting:*** Violence detection and notification
- ***Unified monitoring:*** Metrics and log aggregation
- ***Security maintenance:*** Credential rotation and updates

The Jenkins pipeline (Figure 2) implements automatic error recovery through:

- Idempotent Ansible playbooks
- Atomic deployment stages
- Automated rollback on verification failure
- Health-check driven progression

Architectural innovations

ARM64-optimised orchestration

GPU-CPU memory arbitration: We introduced dynamic GPU memory scaling where:

```
gpu_mem =
 128MB if cpu_load > 70%
 256MB otherwise
```

Edge-optimised observability

Metric compression: We implemented Prometheus' chunk encoding reducing bandwidth by 62%:

```
scrape_configs:
• sample_limit: 2000
    metric_relabel_configs:

• regex: '(container_cpu_.*)'
    action: keep
```